

Assignment3

Atomicity:

1. Without atomic counter

Conducted this experiment with a regular integer. Below is the screenshot of the race condition as multiple go routines access the same `ops` variable

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run without_atomic.go
Expected: 50000
Got: 24112
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run without_atomic.go
Expected: 50000
Got: 30185
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run without_atomic.go
Expected: 50000
Got: 23086
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run without_atomic.go
Expected: 50000
Got: 25748
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run without_atomic.go
Expected: 50000
Got: 27154
○ (base) srimsiramesh@Srimansis-Laptop Atomicity %
```

2. With atomic counter

Fixed the race condition using an atomic integer. Below is the screenshot of the result. Since `ops` is an atomic variable the `read-modify-write` happens as a single indivisible step. No other goroutine can interrupt it.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
• (base) srimsiramesh@Srimansis-Laptop Assignments % cd Assignment3/Atomicity
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run with_atomic.go
ops: 50000
• (base) srimsiramesh@Srimansis-Laptop Atomicity % go run with_atomic.go
ops: 50000
○ (base) srimsiramesh@Srimansis-Laptop Atomicity %
```

3. With -race flag

The `-race` flag enables Go's built-in race detector. It finds race conditions at runtime.

1. Without atomic counter - The race condition is clearly flagged

```

ops: 50000
(base) srimsiramesh@Srimansis-Laptop Atomicity % go run -race without_atomic.go
=====
WARNING: DATA RACE
Read at 0x00c000114038 by goroutine 8:
    main.main.func1()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:21 +0x88

Previous write at 0x00c000114038 by goroutine 7:
    main.main.func1()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:21 +0x98

Goroutine 8 (running) created at:
    main.main()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:18 +0x70

Goroutine 7 (finished) created at:
    main.main()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:18 +0x70
=====
WARNING: DATA RACE
Write at 0x00c000114038 by goroutine 8:
    main.main.func1()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:21 +0x98

Previous write at 0x00c000114038 by goroutine 9:
    main.main.func1()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:21 +0x98

Goroutine 8 (running) created at:
    main.main()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:18 +0x70

Goroutine 9 (running) created at:
    main.main()
        /Users/srimsiramesh/Documents/BSDS/Assignments/Assignment3/Atomicity/without_atomic.go:18 +0x70
=====
Expected: 50000
Got:      44722
Found 2 data race(s)
exit status 66
(base) srimsiramesh@Srimansis-Laptop Atomicity %

```

2. With atomic counter

```

EXIT STATUS 66
(base) srimsiramesh@Srimansis-Laptop Atomicity % go run -race with_atomic.go
ops: 50000

```

Collections:

1. Without mutex:

Without the mutex the program crashes. If two goroutines simultaneously access a single map/data, they can:

- Corrupt the internal bucket structure
- Overwrite each other's pointers
- Cause infinite loops in the hash table
- Trigger Go's built-in race detector crash

Go intentionally crashes when it detects concurrent map writes to prevent silent data corruption.

```
⊗ (base) srinansiramesh@Srimansis-Laptop Collections % go run without_mutex.go
fatal error: concurrent map writes

goroutine 13 [running]:
internal/runtime/maps.fatal({0x1047793eb?, 0x0?})
    /usr/local/go/src/runtime/panic.go:1046 +0x20
main.main.func1(0x6)
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:21 +0x68
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 1 [runnable]:
sync.(*WaitGroup).Add(0x140000100b0, 0x1)
    /usr/local/go/src/sync/waitgroup.go:77 +0x274
main.main()
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:16 +0x6c

goroutine 7 [running]:
goroutine running on other thread; stack unavailable
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 8 [runnable]:
main.main.gowrap1()
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17
runtime.goexit({})
    /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 9 [runnable]:
internal/runtime/maps.(*table).uncheckedPutSlot(0x14000182080?, 0x1047b0a80?, 0xa0c4aca112965874?, 0x1400018c008?, 0x1400018c010?)
    /usr/local/go/src/internal/runtime/maps/table.go:381 +0x1b8
internal/runtime/maps.(*table).grow(0x14000182060, 0x1047b0a80, 0x140001020f0, 0x80?)
    /usr/local/go/src/internal/runtime/maps/table.go:1224 +0x190
internal/runtime/maps.(*table).rehash(0x140001020f0?, 0x1047b0a80?, 0x0?)
    /usr/local/go/src/internal/runtime/maps/table.go:1136 +0x30
main.main.func1(0x2)
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:21 +0x68
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 10 [runnable]:
main.main.gowrap1()
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17
runtime.goexit({})
    /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 11 [runnable]:
main.main.gowrap1()
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17
runtime.goexit({})
    /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
    /Users/srinansiramesh/Documents/BSDS/Assignments/Assignment3/Collections/without_mutex.go:17 +0x4c

goroutine 12 [runnable]:
```

2. With Mutex:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
● (base) sriransiramesh@Sriransis-Laptop Collections % go run mutex.go
=== MUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 11.556542ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run mutex.go
=== MUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 14.193125ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run mutex.go
=== MUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 6.462958ms
○ (base) sriransiramesh@Sriransis-Laptop Collections %
```

3. RWMutex:

RWMutex (Read-Write Mutex) allows multiple readers OR one writer at a time.

```
● (base) sriransiramesh@Sriransis-Laptop Collections % go run RWMutex.go
=== RWMUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 7.172208ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run RWMutex.go
=== RWMUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 11.908375ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run RWMutex.go
=== RWMUTEX PROTECTED MAP ===
Expected length: 50000
Actual length: 50000
Time taken: 6.3055ms
○ (base) sriransiramesh@Sriransis-Laptop Collections %
```

4. Sync Map:

sync.Map is Go's built-in concurrent map, designed for specific use cases without needing explicit locks.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

Time taken:      6.3055ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run SyncMap.go
=== SYNC.MAP ===
Expected length: 50000
Actual length:   50000
Time taken:      7.619334ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run SyncMap.go
=== SYNC.MAP ===
Expected length: 50000
Actual length:   50000
Time taken:      5.698833ms
● (base) sriransiramesh@Sriransis-Laptop Collections % go run SyncMap.go
=== SYNC.MAP ===
Expected length: 50000
Actual length:   50000
Time taken:      3.01025ms
○ (base) sriransiramesh@Sriransis-Laptop Collections % █
```

Analysis:

1. sync.Map is optimized for two specific scenarios: when keys are written once and read many times, and when multiple goroutines access disjoint (non-overlapping) keys. In our test cases, each goroutine writes to unique keys (`g*1000 + i`), meaning there's minimal contention between goroutines since they're not fighting over the same keys. sync.Map handles this efficiently through internal sharding, which allows concurrent writes to different parts of the map without blocking each other.
2. In this write-only experiment, RWMutex and Mutex should theoretically have similar performance since both require exclusive locks for write operations. In a write-heavy workload, RWMutex offers no real benefit over Mutex because writes still require exclusive access. The true advantage of RWMutex would only appear in read-heavy workloads, where multiple readers can access the map simultaneously without blocking each other.

Tradeoffs Summary

Approach	Pros	Cons	Best Use Case
Mutex	Simple, predictable	Slowest, blocks all	Simple cases, low contention
RWMutex	Fast reads, familiar API	Write still blocks all	Read-heavy workloads

Approach	Pros	Cons	Best Use Case
sync.Map	Fastest, no explicit locking	Different API, less flexible	Disjoint keys, write-once read-many

File Access

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS

• (base) sriramansiramesh@Srimansis-Laptop File_Access % go run file_access.go
=== FILE ACCESS EXPERIMENT ===
Writing 100000 lines to file

Unbuffered: 138.521542ms
Buffered: 7.708042ms
• (base) sriramansiramesh@Srimansis-Laptop File_Access % go run file_access.go
=== FILE ACCESS EXPERIMENT ===
Writing 100000 lines to file

Unbuffered: 139.956333ms
Buffered: 7.696916ms
• (base) sriramansiramesh@Srimansis-Laptop File_Access % go run file_access.go
=== FILE ACCESS EXPERIMENT ===
Writing 100000 lines to file

Unbuffered: 112.05ms
Buffered: 8.361667ms
○ (base) sriramansiramesh@Srimansis-Laptop File_Access %

```

Analysis:

1. Unbuffered Write:

Each `f.Write()` call triggers a system call to the operating system, which then writes to disk. For 100,000 lines, that's 100,000 system calls.

System calls are expensive because they involve:

- Context switch from user mode to kernel mode
- OS handling the I/O request
- Waiting for disk
- Context switch back to user mode

2. Buffered Write:

The buffer collects writes in memory and only writes to disk when the buffer is full.

Tradeoffs

Aspect	Unbuffered	Buffered
Speed	Slow	Fast
Memory usage	Lower	Higher (buffer in RAM)
Data safety	Immediate write to disk	Data could be lost if crash before flush
Simplicity	Simpler code	Requires flush/close handling

Context Switching

```
● (base) sriramesh@Sriramesh-Laptop: ~/Assignments/4-ed/context_switching
● (base) sriramesh@Sriramesh-Laptop: Context_Switching % go run context_switch.go
=== CONTEXT SWITCHING EXPERIMENT ===
Ping-pong iterations: 1000000
Total switches: 2000000 (2 per round-trip)

GOMAXPROCS: 1 (single OS thread)
Total time:    183.454084ms
Avg switch:    91.73 ns

GOMAXPROCS: 8 (multiple OS threads)
Total time:    229.484958ms
Avg switch:    114.74 ns

=== COMPARISON ===
Single thread is 1.25x faster
● (base) sriramesh@Sriramesh-Laptop: Context_Switching % go run context_switch.go
=== CONTEXT SWITCHING EXPERIMENT ===
Ping-pong iterations: 1000000
Total switches: 2000000 (2 per round-trip)

GOMAXPROCS: 1 (single OS thread)
Total time:    173.71075ms
Avg switch:    86.86 ns

GOMAXPROCS: 8 (multiple OS threads)
Total time:    225.886833ms
Avg switch:    112.94 ns

=== COMPARISON ===
Single thread is 1.30x faster
● (base) sriramesh@Sriramesh-Laptop: Context_Switching % go run context_switch.go
=== CONTEXT SWITCHING EXPERIMENT ===
Ping-pong iterations: 1000000
Total switches: 2000000 (2 per round-trip)

GOMAXPROCS: 1 (single OS thread)
Total time:    164.553459ms
Avg switch:    82.28 ns

GOMAXPROCS: 8 (multiple OS threads)
Total time:    226.81925ms
Avg switch:    113.41 ns

=== COMPARISON ===
Single thread is 1.38x faster
● (base) sriramesh@Sriramesh-Laptop: Context_Switching %
```

Analysis:

Single thread is consistently faster (~1.31x) because:

1. User-Space vs Kernel-Space Switching

With GOMAXPROCS=1, both goroutines run on the same OS thread. Switching between them is handled entirely by Go's scheduler in user-space - no operating system involvement needed.

With GOMAXPROCS=8, goroutines may run on different OS threads. Channel communication across threads requires kernel-level synchronization, which is more expensive.

2. Cache Locality

Single thread keeps data in the same CPU core's cache. Multiple threads may cause data to move between CPU cores, resulting in cache misses and memory synchronization overhead.

3. No Cross-Core Communication

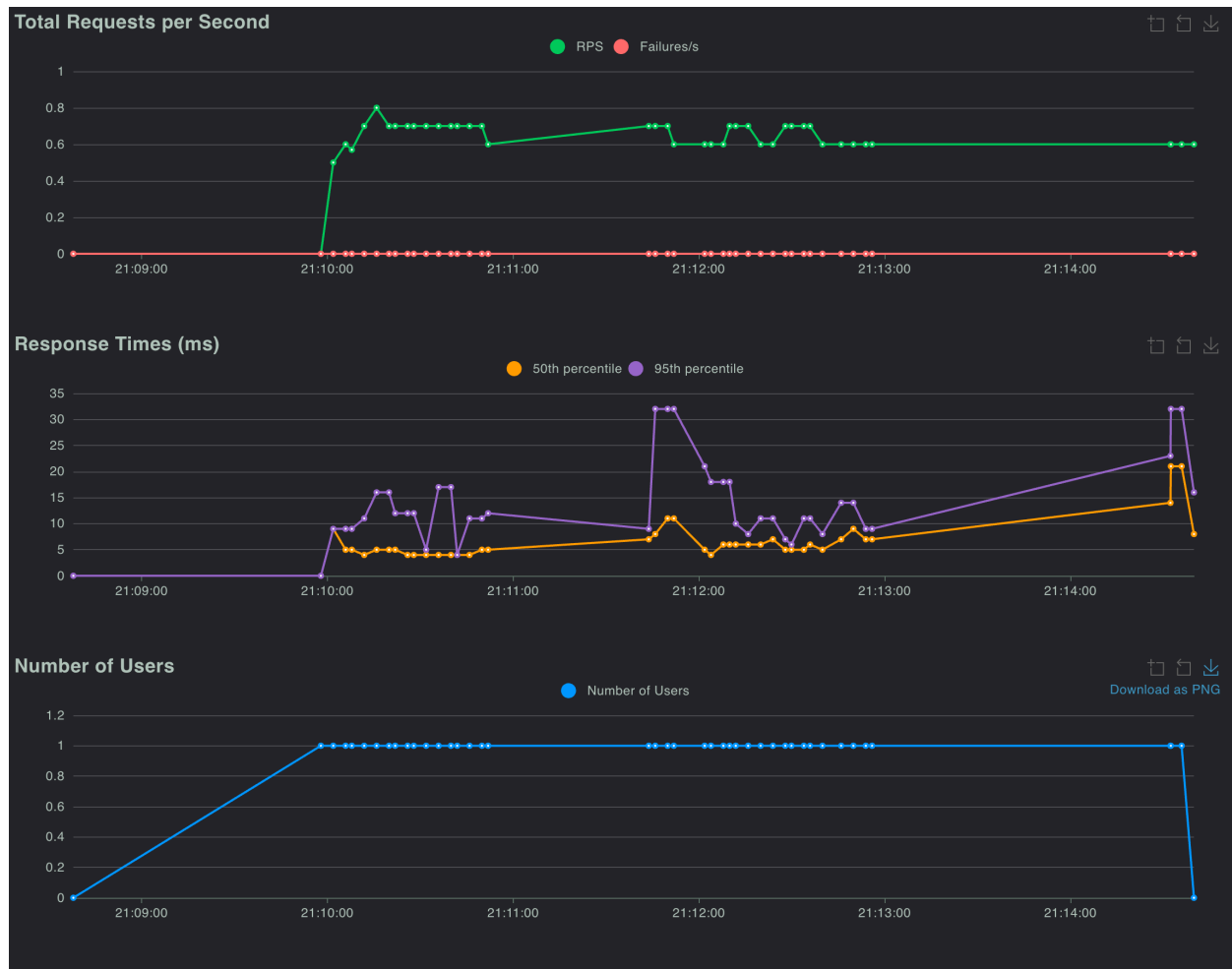
Single thread: Everything happens on one core → no inter-core latency.

Multiple threads: Cores must communicate through shared memory, adding latency.

Locust Testing

1. Without mutex

Charts:



Results:

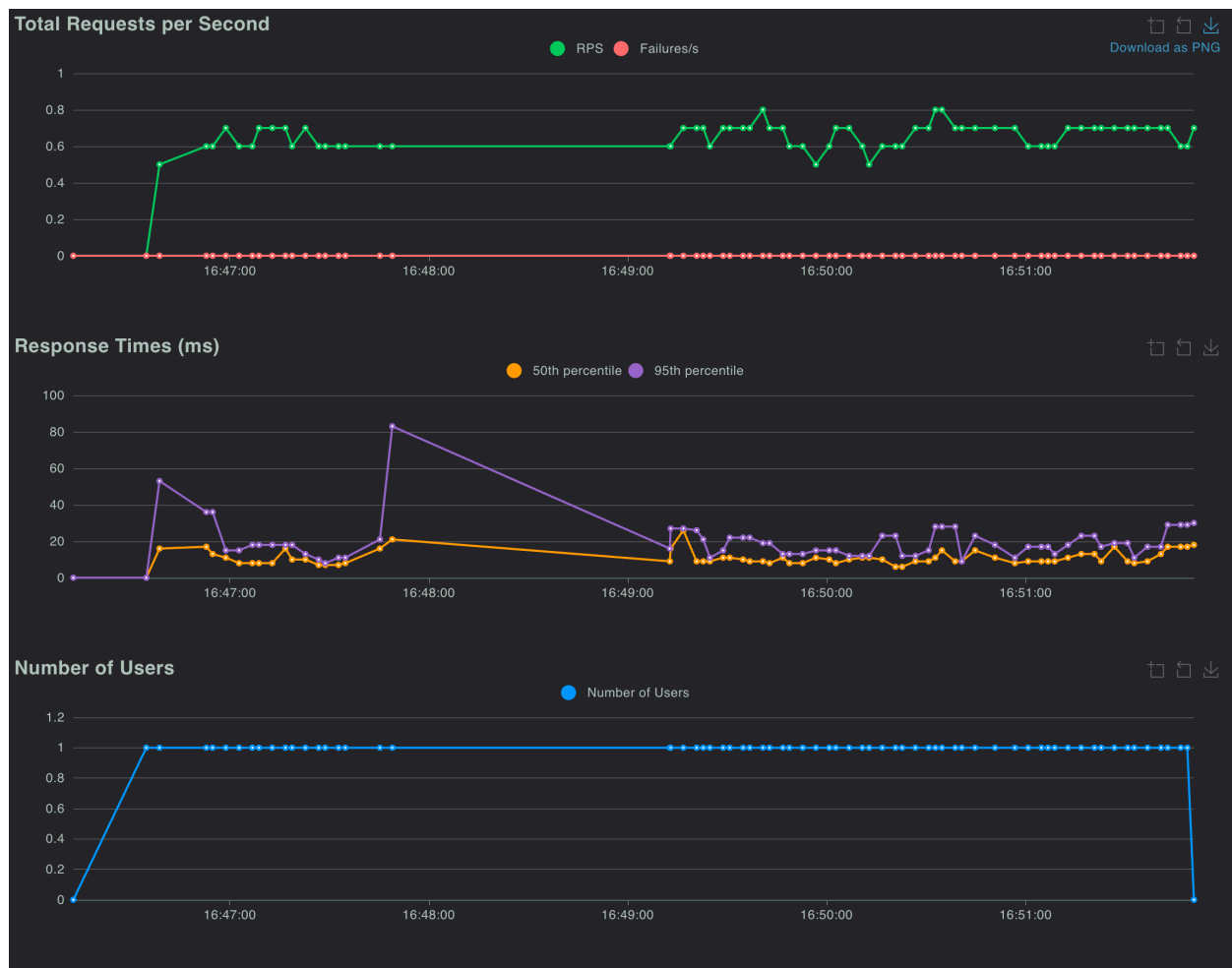
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	89	0	5	18	32	6.92	2	32	2231.89	0.4	0
POST	/albums	31	0	6	17	64	9.77	2	64	94	0.1	0
GET	/albums/1	62	0	6	24	57	9.98	3	57	95	0.1	0
Aggregated		182	0	6	20	57	8.45	2	64	1139.79	0.6	0

2. With mutex:

Results:

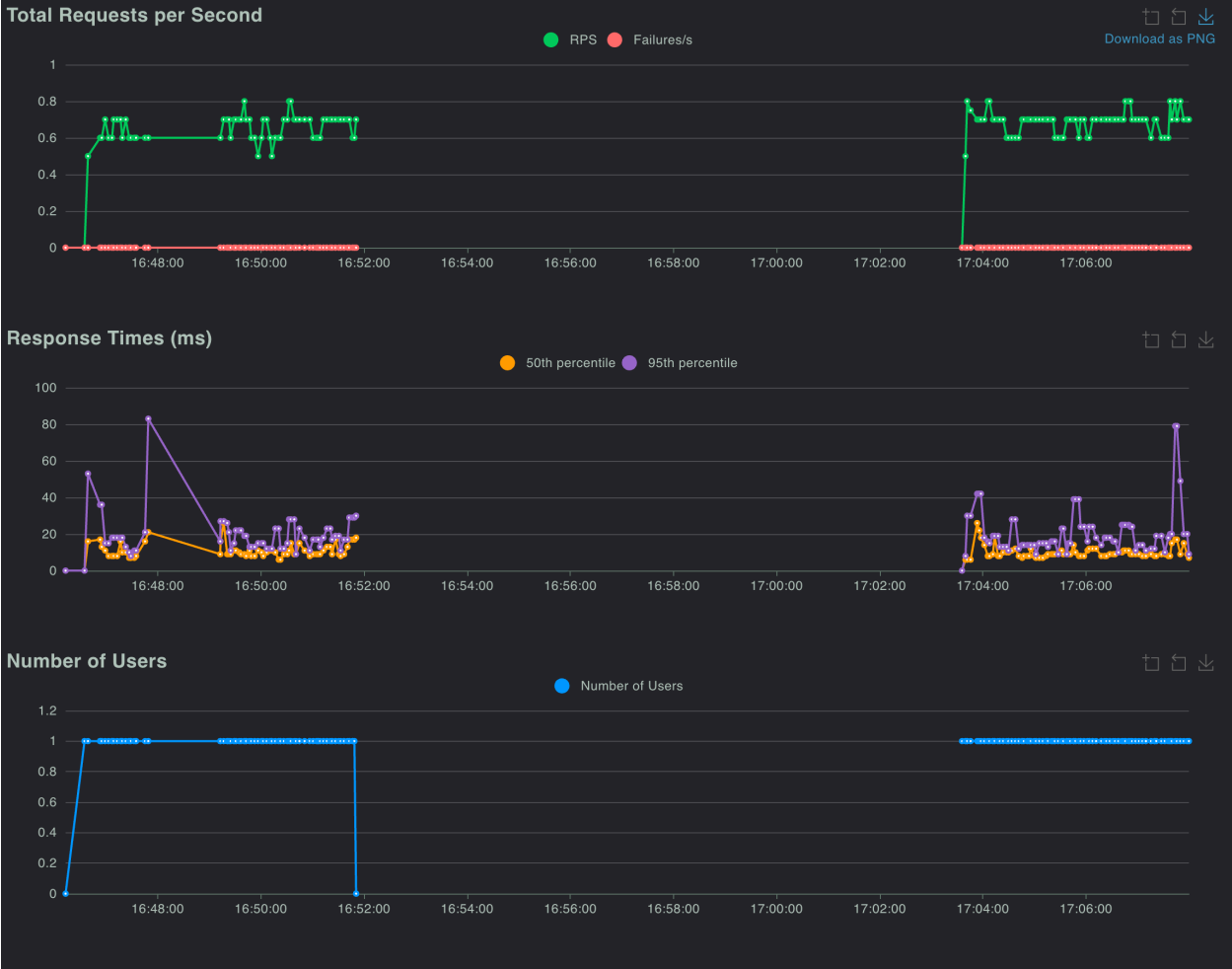
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	103	0	11	28	83	15.27	4	219	2456.95	0.2	0
POST	/albums	35	0	9	32	36	11.05	3	36	94	0.3	0
GET	/albums/1	69	0	9	29	100	12.94	4	103	95	0.2	0
Aggregated		207	0	9	29	83	13.78	3	219	1270.1	0.7	0

Charts:



3. With sync map

Charts:



Results:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	90	0	8	24	54	10.97	2	54	499.33	0.7	0
POST	/albums	25	0	9	30	42	12.58	2	42	94	0	0
GET	/albums/1	50	0	9	20	24	10.06	3	24	95	0.1	0
Aggregated		165	0	9	24	42	10.94	2	54	315.39	0.8	0

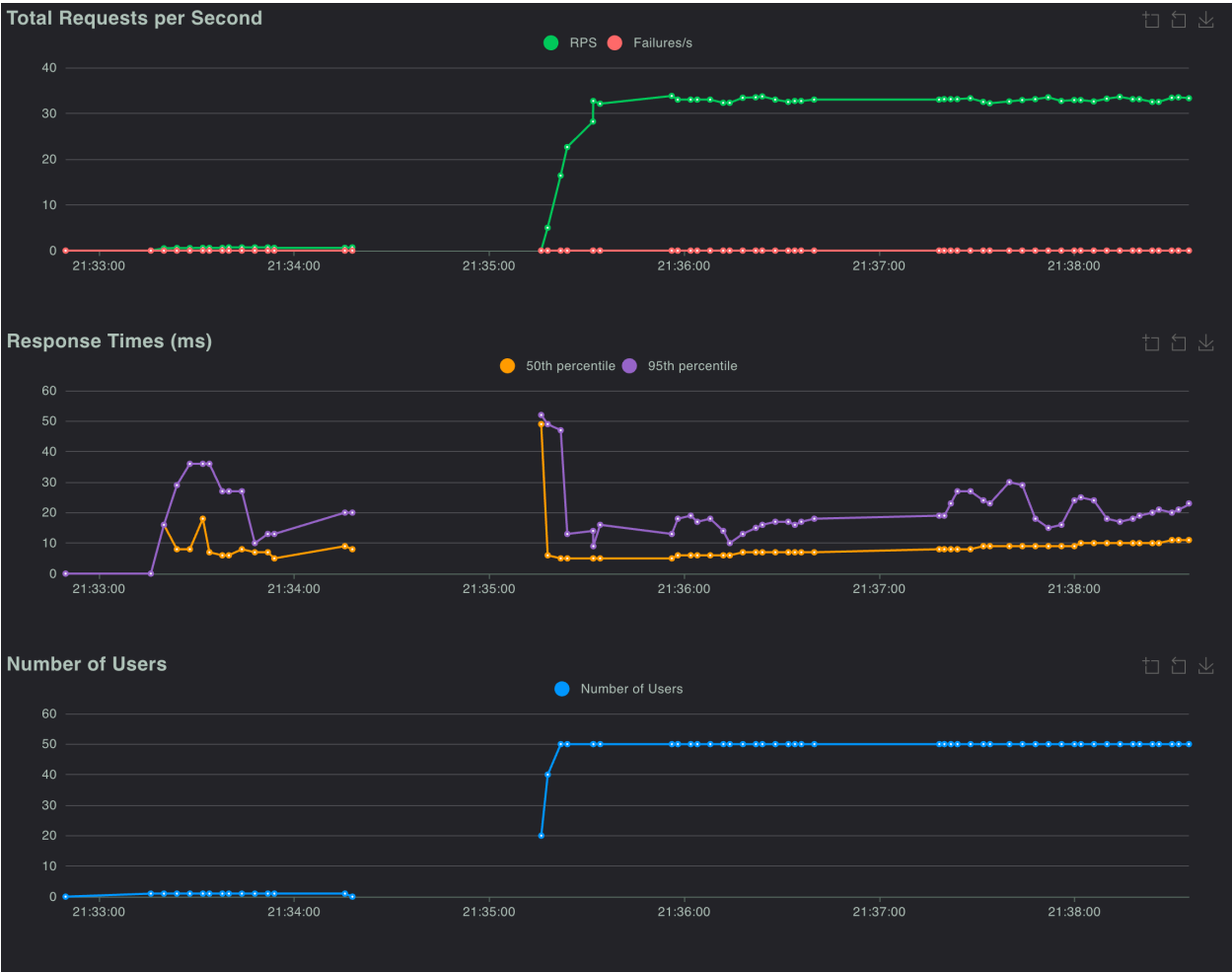
Analysis:

Tradeoffs Summary

Server	Speed	Thread Safety	Complexity	Best For
Original	Fastest	Unsafe	Simple	Never use in production!
Mutex	Medium	Safe	Medium	Read-heavy, familiar API
sync.Map	Fast	Safe	Medium	Concurrent access, fast lookups

Load testing:

Charts:



Results:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	5176	0	9	22	52	10.7	1	192	111421.81	24.4	0
POST	/albums	1745	0	4	14	49	6.03	1	177	94	8.3	0
Aggregated		6921	0	7	21	50	9.52	1	192	83352.59	32.7	0

Docker stats:

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	COMMENTS			
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS	
84451750f822	load-test-locust-worker-1	0.32%	38.79MiB / 3.827GiB	0.99%	634MB / 9.43MB	0B / 0B	3	
24663ea7cfd1	load-test-locust-master-1	0.24%	51.02MiB / 3.827GiB	1.30%	485kB / 2.39MB	938kB / 0B	3	

Analysis:

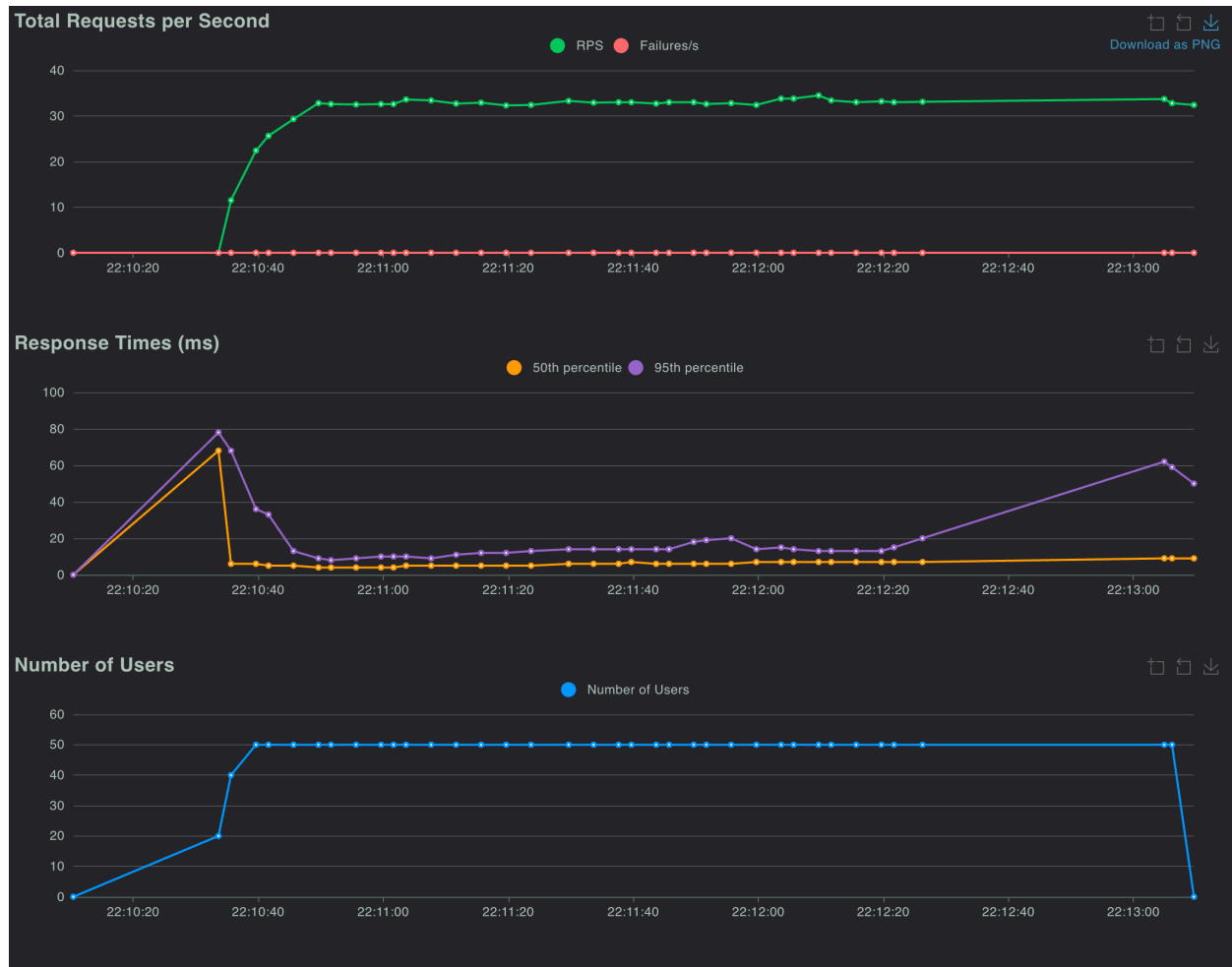
This load test demonstrates that the server handles 50 concurrent users at 32.7 RPS with zero failures. GET operations are slower than POST due to response payload size growing with each new album added. The system shows stable performance characteristics with no degradation over time.

Amdahl's law:

Results:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	2699	0	6	15	33	7.47	1	115	51019.2	25.2	0
POST	/albums	849	0	4	9	20	4.79	1	52	94	7.8	0
Aggregated		3548	0	6	14	30	6.83	1	115	38833.32	33	0

Charts:

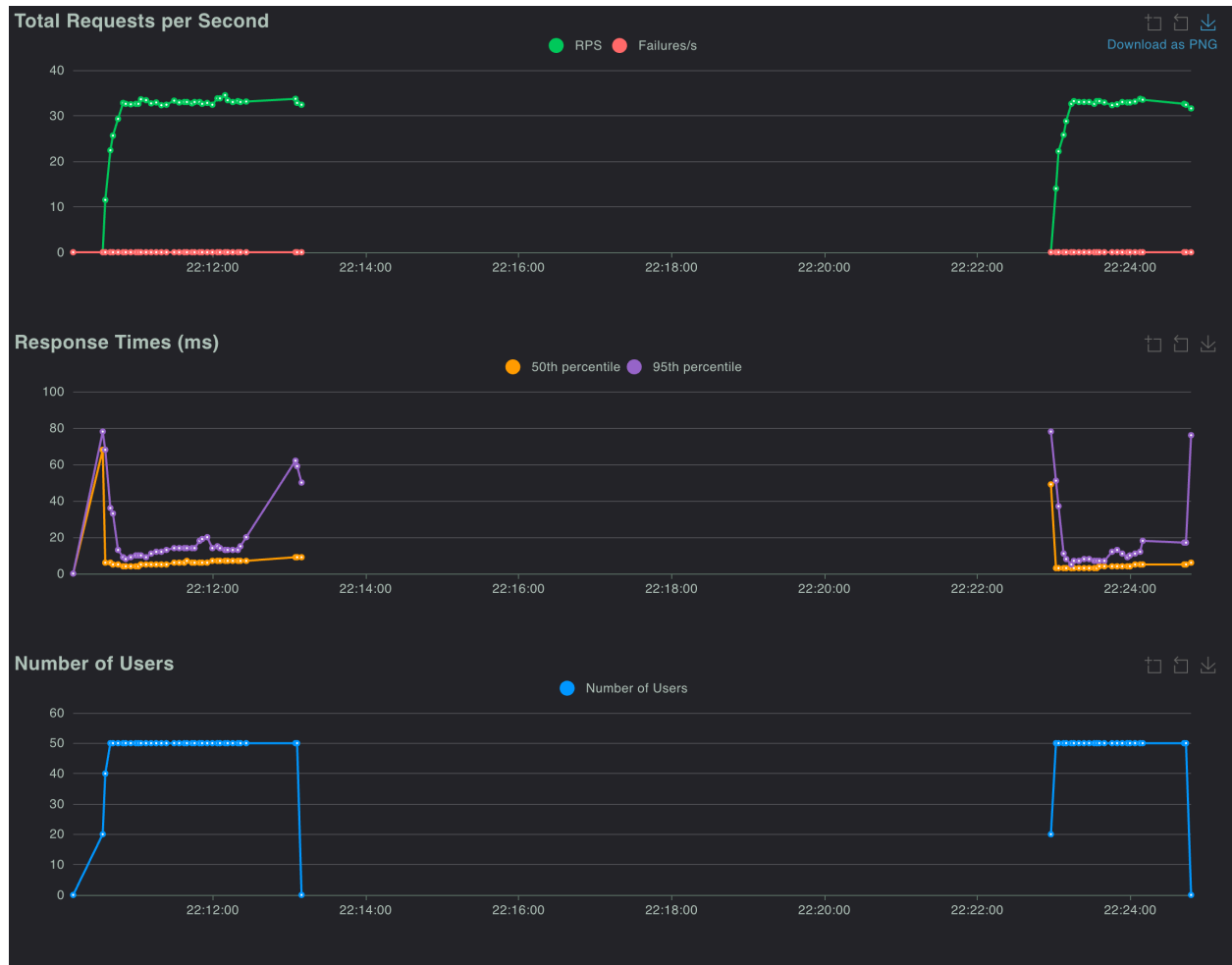


Analysis:

The test with 4 workers achieved approximately 33 RPS, which is nearly identical to the 1 worker test (~32.7 RPS), demonstrating that adding more Locust workers does not linearly increase throughput. This is Amdahl's Law in action - the bottleneck is the single Go server, not the load generator, so parallelizing the client side provides no benefit. If the server used a shared hashmap with mutex protection, the reading and writing contention would further limit throughput since writes require exclusive locks that block all other operations - this serial portion is what Amdahl's Law identifies as the fundamental limit to parallelization.

Context Switching:

Charts:



Results:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	2654	0	4	16	76	7.51	1	329	53251.63	23.9	0
POST	/albums	890	0	2	8	51	4.07	1	274	94	7.7	0
Aggregated		3544	0	4	14	69	6.65	1	329	39902.23	31.6	0

Analysis:

FastHttpUser achieved lower response times than HttpUser, with GET median dropping from 6 ms to 4 ms and POST median dropping from 4 ms to 2 ms. This improvement happens because FastHttpUser uses a C-based HTTP client (geventhttpclient) which is faster and more lightweight than Python's default requests library. However, the total RPS remained similar at around 31-33, confirming that the Go server is the bottleneck, not Locust. FastHttpUser gives us more accurate server performance measurements by reducing client-side overhead.